

PoC – Migração e consultas de Logs para o MongoDB

1. Objetivo

Avaliar a viabilidade técnica para migração de logs para o MongoDB e o desempenho de consultas, comparando os dois bancos de dados nos seguintes aspectos:

- Tempo de execução das consultas;
- Custo de carga e preparação dos dados (ingestão e desnormalização);
- Escalabilidade com diferentes volumes de dados.

2. Contexto

A tabela **LOG_2025** no Oracle possui atualmente **2.5 bilhões** de registros. O crescimento contínuo desse volume tem impactado o desempenho das consultas e representa um desafio crescente de armazenamento, tornando necessária a avaliação de alternativas.

O MongoDB foi selecionado para esta *Proof of Concept* (PoC) por sua arquitetura orientada a documentos, que permite a desnormalização de dados e a criação de índices sobre subdocumentos e campos derivados, características relevantes para o perfil de consultas analíticas realizadas sobre os logs.

3. Escopo

Os seguintes pontos estão dentro do escopo desta PoC:

- Execução de benchmarks de consulta em Oracle 19c e MongoDB;
- Comparação de desempenho entre a abordagem relacional (Oracle) e a abordagem orientada a documentos com desnormalização (MongoDB);
- Mensuração do tempo de carga de dados no MongoDB (ingestão via MongolImport);

- Mensuração do tempo de preparação dos dados desnormalizados (atualização em massa dos campos derivados).

4. Ambientes e Ferramentas Utilizadas

	Oracle (Oracle 19C Enterprise Edition - Docker) , banco de dados relacional utilizado para replicar a estrutura e os dados de produção em ambiente local.
	MongoDB (MongoDB Community - Docker) , banco de dados NoSQL orientado a documentos, selecionado para os testes de viabilidade de migração e consulta de logs.
	Docker , utilizado para orquestração dos contêineres, provendo um ambiente local isolado com ambos os bancos de dados e as dependências necessárias para a PoC.
	SQL Developer , utilizado para visualização do banco Oracle, execução dos benchmarks manuais, leitura de planos de execução e inspeção de tabelas e índices.
	NoSQLBooster , utilizado para visualização das collections e índices do MongoDB.
	Linguagem de programação Python , linguagem utilizada para desenvolvimento dos scripts de automação de carga, atualização em massa dos campos desnormalizados e execução dos benchmarks em ambos os bancos de dados.

Bibliotecas utilizadas com Python: **Pandas**, **PyMongo**, **SQLAlchemy**, **OracleDB**.

5. Restrições

- O volume total do banco de dados em produção inviabiliza testes locais com a base de dados completa;
- O hardware utilizado pode não refletir o desempenho observado em ambiente de produção, onde servidores dedicados tendem a apresentar resultados mais estáveis e expressivos;
- O SSD/HDD utilizado é de uso geral (estação de trabalho), sem configuração RAID ou volumes dedicados por banco de dados em produção.

6. Bases de Dados Utilizadas

Para as análises foram utilizadas bases de dados com volumes de 3, 10 e 50 milhões de tuplas/documentos. A base de 50 milhões foi gerada a partir da replicação da base de 10 milhões, mantendo a mesma distribuição de valores e estrutura dos dados, com o objetivo de avaliar o comportamento de escalabilidade de cada banco de dados em volumes elevados sem a necessidade de extração da base de produção.

7. Estratégia de Modelagem no MongoDB

A estratégia de modelagem adotada consiste em desnormalizar no documento MongoDB os dados que, no Oracle, exigem JOIN ou leitura de campos CLOB. O campo **log_tipo_detalhes** incorpora diretamente os atributos da tabela **LOG_TIPO** relacionada a cada registro, eliminando a necessidade de junção em tempo de consulta. O campo **valor_novo_detalhes** é derivado do campo CLOB **VALOR_NOVO**, com os dados relevantes extraídos e estruturados como subdocumento. Ambos os campos são populados via scripts de atualização em massa ([scripts](#)), executados após a carga inicial dos documentos.

```

{
  "_id" : ObjectId("6a0b3d1ca7147a8efd6262a7"),
  "ID_LOG" : Long("7030343277"),
  "ID_USU" : 279008,
  "IP_COMPUTADOR" : "177.124.60.75",
  "DATA" : ISODate("2025-01-01T21:00:00.000-03:00"),
  "HORA" : ISODate("2025-01-01T21:00:00.000-03:00"),
  "TABELA" : "Arquivo",
  "VALOR_ATUAL" : "PDF COMPLETO [Origem:]",
  "CODIGO_TEMP" : 79003,
  "ID_TABELA" : Double("253537572"),
  "HASH" : null,
  "QTD_ERROS_DIA" : null,
  "log_tipo_detalhes" : {
    "LOG_TIPO_CODIGO" : "9",
    "LOG_TIPO" : "Download",
    "CODIGO_TEMP" : null,
    "STATUS" : "1"
  },
  "valor_novo_detalhes" : {
    "Id_Arquivo" : "Id Arquivo",
    "NomeArquivo" : "NomeArquivo.pdf",
    "Id_ArquivoTipo" : "Id_ArquivoTipo",
    "ArquivoTipo" : "ArquivoTipo",
    "ContentType" : "application/pdf",
    "Caminho" : "Caminho",
    "DataInsercao" : "23/08/2022 14:10:11",
    "UsuarioAssinador" : "C=BR,O=ICP-Brasil,OU=Secretaria da Receita Federal do Brasil - RFB,OU=RFB e-CPF A3,OU=VALID,OU=AR CERTDATA,OU=16986332000127,CN=UsuarioAssinador",
    "CodigoTemp" : "CodigoTemp",
    "ArquivoTipoCodigo" : "ArquivoTipoCodigo",
    "Origem" : ""
  }
},

```

Como benefícios esperados através dessas atualizações é esperado:

- Redução do tempo de resposta, especialmente em consultas sobre campos derivados de CLOB;
- Eliminação de JOINS em tempo de consulta, simplificando os pipelines de agregação.
- Suporte a índices diretos em subdocumentos e campos extraídos de CLOB;
- Documento autossuficiente, reduzindo dependência de tabelas auxiliares;
- Maior legibilidade e autodescrição dos documentos armazenados.

8. Consultas Avaliadas

As consultas têm como objetivo serem equivalentes entre si, portanto, por mais que a sintaxe seja diferente o resultado das consultas tendem a ser as mesmas.

ID	Descrição
Q01	DISTINCT com filtro por tipo de log e data
Q02	COUNT simples com filtro de data
Q03	Filtro por usuário
Q04	Filtro por IP
Q05	Filtro composto: data, tipo de log e tabela
Q06	Agregação COUNT por IP
Q07	Filtro por nome do tipo de log
Q08	Paginação: 1000 registros mais recentes
Q09	Busca por substring em VALOR_ATUAL
Q10	Agregação temporal: COUNT por dia
Q11~Q15	Consultas variadas sobre campos desnormalizados (50M)

A especificação completa das consultas, incluindo pipelines e queries, está disponível em: [link](#).

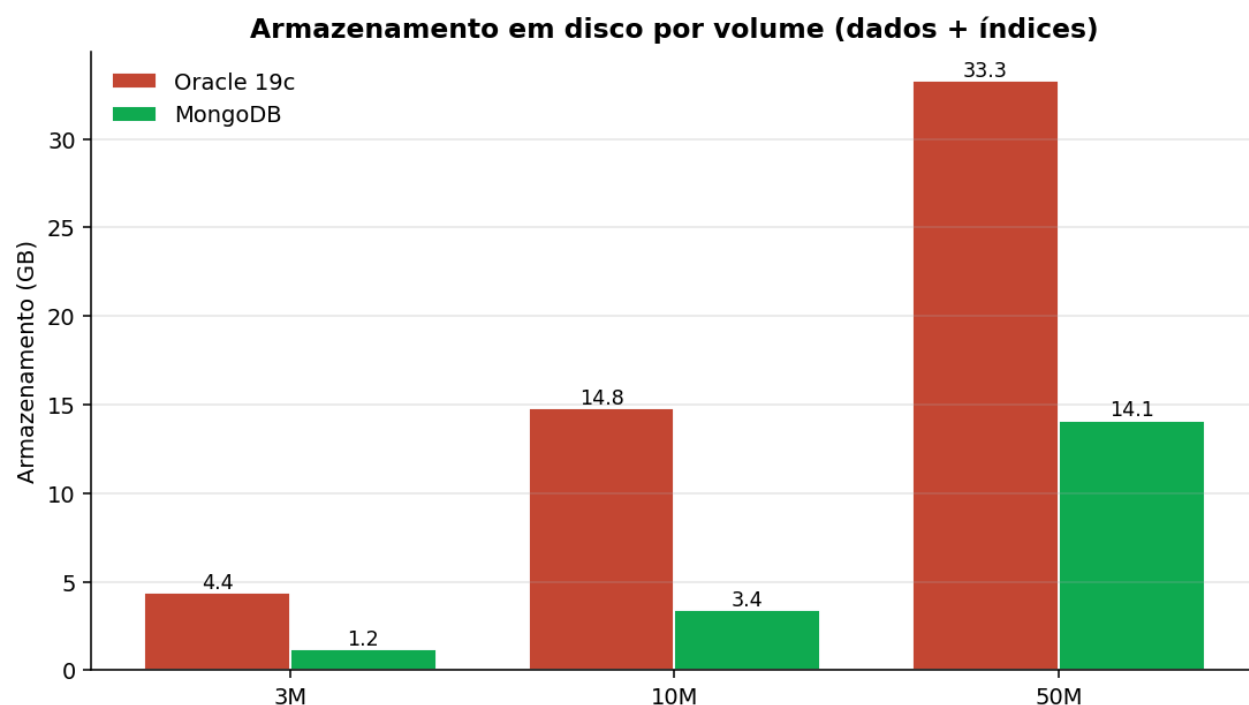
9. Benchmark's

Armazenamento

No MongoDB, as collections contendo 3, 10 e 50 milhões de documentos, incluindo seus respectivos índices, apresentaram tamanhos de **1,2 GB**, **3,4 GB** e **14,1 GB**, totalizando **18,7 GB** para toda a base de dados.

No Oracle, os volumes equivalentes de 3, 10 e 50 milhões de tuplas ocuparam **4,38 GB**, **14,77 GB** e **33,28 GB**, totalizando **52,43 GB** para toda a base de dados.

O MongoDB apresentou consumo de armazenamento significativamente menor em todos os volumes testados, em média **64%** inferior ao Oracle, com a diferença se acentuando conforme o volume cresce: no volume de 50 milhões, a collection ocupou aproximadamente **42%** do espaço utilizado pela tabela equivalente no Oracle.



Inserts

Os dados utilizados foram extraídos do banco de produção via SQL Developer e exportados em formato **.csv**, sendo utilizados como fonte tanto para a carga no MongoDB quanto no Oracle em ambiente Docker.

MongoDB (carga via Python/PyMongo)

Volume	Tempo	Throughput
3 Milhões	102,1 segundos	29.384 doc/s
10 Milhões	371,1 segundos	26.905 doc/s
150 Milhões *	2.889 segundos	51.921 doc/s

** Teste complementar realizado com **bulkWrite** em lotes, não incluído nos volumes do benchmark principal. O teste equivalente não foi realizado no Oracle pois o volume de 150 milhões de tuplas demandaria aproximadamente 100 GB de armazenamento, inviabilizando a execução no ambiente local disponível.*

Oracle (carga via Python/SQLAlchemy/Pandas)

Volume	Tempo	Throughput
3 Milhões	1.136,7 segundos	2.639 reg/s
10 Milhões	2.933,8 segundos	3.409 reg/s

Updates (MongoDB)

Para viabilizar a estratégia de desnormalização, foram realizados dois processos de atualização em massa nas collections do MongoDB: o primeiro, responsável pela criação do campo **log_tipo_detalhes**, incorporando os atributos da tabela **LOG_TIPO** diretamente em cada documento; o segundo, pela criação do campo **valor_novo_detalhes**, extraindo e estruturando os dados relevantes do campo CLOB **VALOR_NOVO**.

Volume	Campo	Tempo (s)	Throughput
3 Milhões	log_tipo_detalhes	75,5 segundos	39.737 doc/s
3 Milhões	valor_novo_detalhes	89,9 segundos	36.187 doc/s
10 Milhões	log_tipo_detalhes	251,3 segundos	39.792 doc/s
10 Milhões	valor_novo_detalhes	338,1 segundos	29.577 doc/s

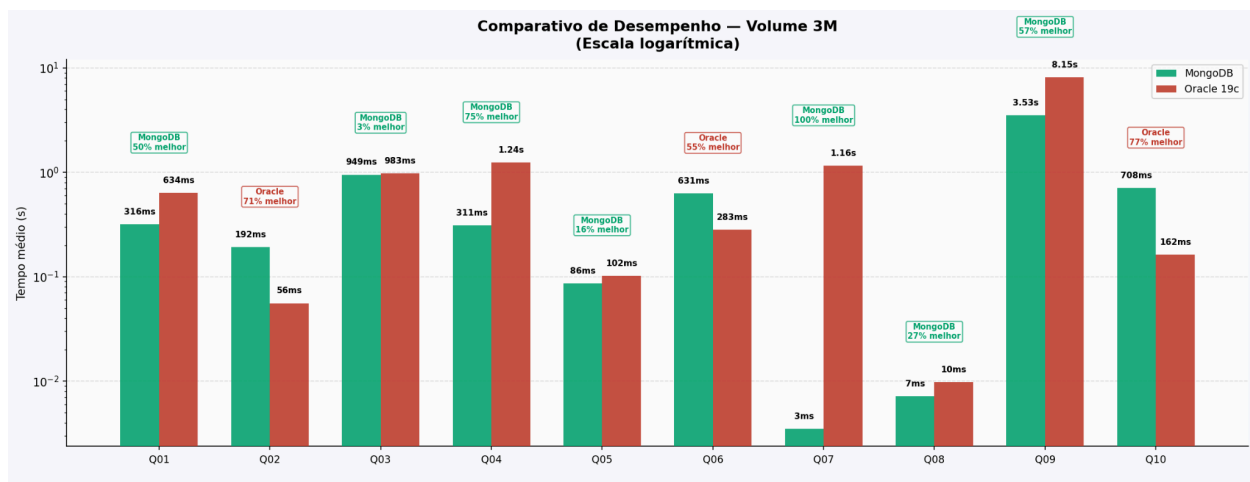
** Os tempos acima representam um custo único de preparação, executado após a carga inicial dos dados.*

Consultas

As consultas foram executadas em ambos os bancos de dados nos volumes de 3, 10 e 50 milhões de registros, totalizando 10 consultas por volume e 5 consultas adicionais exclusivas para o volume de 50 milhões, estas últimas elaboradas especificamente para avaliar o desempenho sobre os campos desnormalizados (**log_tipo_detalhes** e **valor_novo_detalhes**).

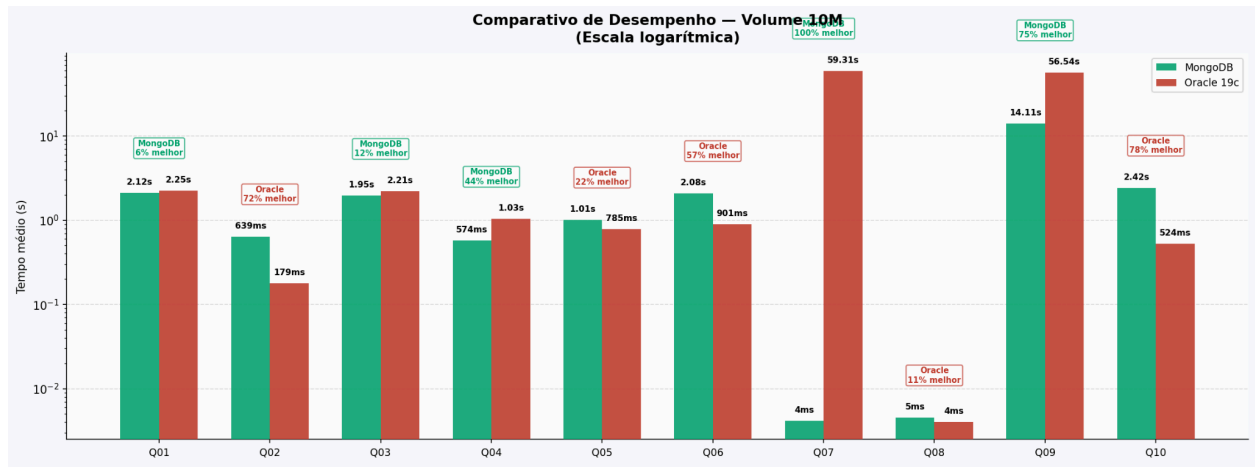
Cada consulta foi executada 7 vezes, sendo descartada a execução identificada como outlier pelo método **IQR**, trabalhando portanto com no mínimo 6 execuções válidas por consulta.

Volume 3M



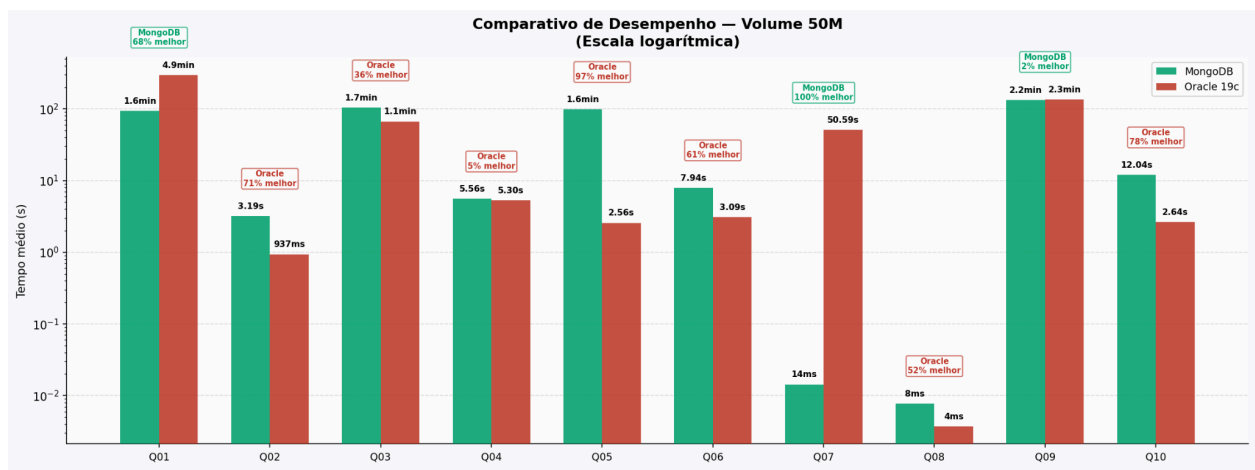
Oracle venceu em COUNT simples (Q02), agregação por IP (Q06) e agregação temporal (Q10). MongoDB venceu nas demais, com destaque para a Q07 (filtro por nome do tipo de log) executando em **3ms** contra **1,16s** do Oracle, reflexo direto da desnormalização do campo **log_tipo_detalhes**.

Volume 10M

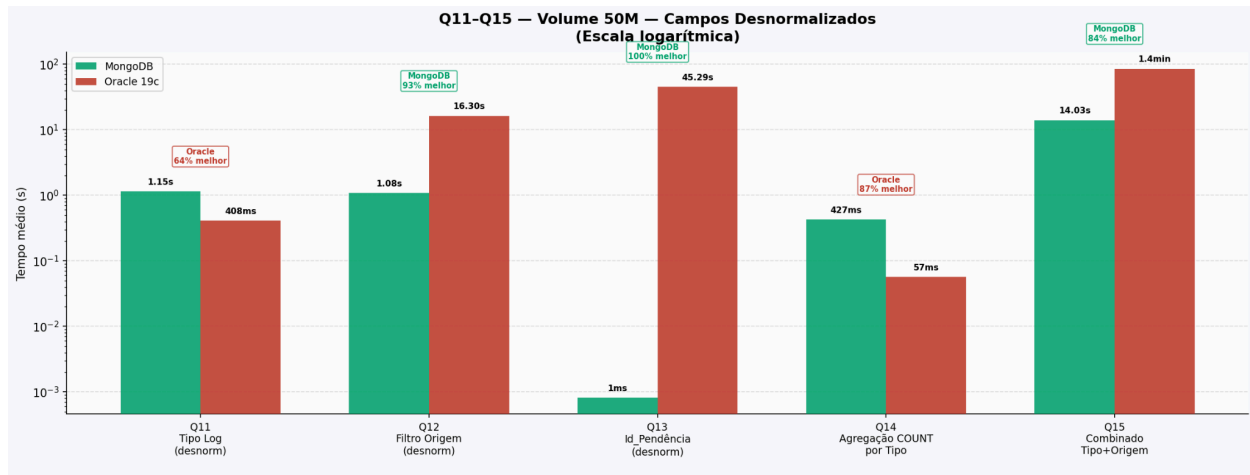


Oracle manteve vantagem nas agregações e COUNT simples. A Q07 se acentuou: MongoDB em **4ms** contra **59,31s** do Oracle. A Q09 (busca por substring) também favoreceu o MongoDB (**14,11s** contra **56,54s**), sugerindo que consultas sobre texto livre escalam melhor no MongoDB.

Volume 50M



Oracle dominou filtros compostos e agregações, com destaque para a Q05 (97% mais rápido). MongoDB venceu em consultas analíticas e sobre campos desnormalizados.



Oracle venceu nas consultas simples sobre campos indexados (Q11, Q14). MongoDB dominou as consultas sobre conteúdo derivado de CLOB: Q12 (93% mais rápido), Q13 (100%) e Q15 (84%), confirmando que o maior ganho da migração está exatamente nessas operações.

10. Conclusão

Com base nos resultados desta PoC:

Viabilidade Técnica: a migração é tecnicamente viável. A estrutura dos logs se adapta bem ao modelo de documentos, e o processo de ingestão e desnormalização mostrou-se executável em tempo aceitável mesmo em ambiente local com hardware limitado.

Ganho de Performance: o ganho é significativo em consultas sobre campos CLOB e JOIN com tabelas auxiliares, Q12, Q13 e Q15, apresentaram vantagens de **84%** a **100%** no MongoDB. O Oracle mantém vantagem em contagens, agregações e filtros compostos sobre colunas indexadas. O ganho é **seletivo** e depende do perfil de consultas em produção.

Custo Operacional: novos documentos precisariam ser ingeridos já desnormalizados, exigindo manutenção dos scripts como parte do pipeline de ingestão. Alterações na tabela **LOG_TIPO** exigiriam atualização em massa nos documentos MongoDB. O ganho de armazenamento (em média 64% menor que o Oracle) é um fator relevante para justificar esse custo, considerando os **2,5 bilhões de registros** em produção.

Sugestões para próximos passos

Caso os resultados desta PoC sejam considerados satisfatórios e a migração faça sentido para o contexto:

- Executar os benchmarks em servidor dedicado com Linux, eliminando o overhead do Docker no Windows;
- Mapear as consultas mais executadas em produção sobre a **LOG_2025** e priorizá-las nos testes;
- Validar os resultados com um subconjunto real da **LOG_2025** em servidor, confirmando se a distribuição de dados em produção reflete os resultados desta PoC.

Todos os dados, consultas e scripts podem ser acessados neste [repositório](#).